

You can control the execution of Octane with some parameters:

- `--host`: Default `127.0.0.1`, the IP address the server should bind to
- `--port`: Default `8000`, the port the server should be available on
- `--workers`: Default `auto`, the number of workers that should be available to handle requests
- `--max-requests`: Default `500`, the number of requests to process before reloading the server

For example, you can launch Octane with just two workers:

```
php artisan octane:start --workers=2
```

=====

If you launch the server, define the maximum number of requests to process (for each worker) before starting the server with the `max-requests` parameter:

```
php artisan octane:start --workers=2 --max-requests=3
```

=====

Manually, you can use an Octane command. If you are using Laravel Sail (so Docker), you have to run the command in the container. The command is as follows:

```
php artisan octane:reload --server=swoole
```

If you are running in a container, you have to use the `sail` command:

```
vendor/bin/sail php artisan octane:reload --server=swoole
```

=====

To track the execution time, we can use the `hrtime()` function.

Note

When you need to track the execution time of a series of instructions, the use of `hrtime()` is recommended since it is a function that returns monotonic timestamps. A monotonic timestamp is a time calculated based on a reference point (thus relative) and is not affected by system date changes such as automatic clock adjustments (NTP or Daylight Savings Time updates).

=====

To use concurrency with tasks

```
use Laravel\Octane\Facades\Octane;
Route::get('/concurrent-task', function () {
    $start = hrtime(true);
    [$result1, $result2] = Octane::concurrently([
        function () {
            sleep(2);
            return 'Hello';
        },
        function () {
            sleep(2);
            return 'World';
        },
    ]);
    $end = hrtime(true);
    return "{$result1} {$result2} in " . ($end - $start) / '.' seconds';
});

=====
```

With Swoole, you can use the **Octane::tick()** function, where you can provide a name (**as the first parameter**) and the function defining a Closure **as the second parameter**.

```
use Laravel\Octane\Facades\Octane;

class AppServiceProvider extends ServiceProvider
{
    /**
     * Bootstrap any application services.
     */
    public function boot(): void
    {
        Octane::tick(
            'simple-ticker', fn () => Log::info("OCTANE TICK", ['timestamp' =>
now()])
        ) ->seconds(10) ->immediate();
    }
}

=====
```

The **Octane::tick()** method returns the `InvokeTickCallable` object that implements a few methods:

- **__invoke()**: A special method used for invoking the `tick()` listener; it is the method responsible for executing the function passed as the second parameter to the `tick()` method.
- **seconds()**: A method for indicating how often the listener should be automatically invoked (via the **__invoke()** method). It accepts an integer parameter in seconds.
- **immediate()**: A method indicating that the listener should be invoked on the first tick (so, as soon as possible).

=====

To obtain the `Cache` instance with the **Octane provider**, you can use **Cache::store('octane')**. Once you have the instance, you can use the **put()** method to store the new value.

=====

For using caching with Octane using the Swoole Tables:

```
use Illuminate\Support\Facades\Cache;
use Illuminate\Support\Facades\Log;
use Illuminate\Support\ServiceProvider;
use Laravel\Octane\Facades\Octane;

class AppServiceProvider extends ServiceProvider
{
    public function boot(): void
    {
        Octane::tick('cache-last-random-number', function() {
            $number = rand(1, 1000);
            Cache::store('octane')->put('last-random-number', $number);
            Log::info("New number in cache: " . $number, ['timestamp' => now()]);
            return;
        }->seconds(10)->immediate());
    }
}
```

=====

Note:

The storage in the Octane cache is not permanent; it is volatile. This means that you can lose the values every time the server is restarted.

```
=====

Route::get('/get-random-number', function () {
    $number = Cache::store('octane')->get('last-random-number', 0);
    return $number;
});

Route::get('/increment-number', function () {
    $number =
        Cache::store('octane')->increment('my-number');
    return $number;
});

Route::get('/decrement-number', function () {
    $number =
        Cache::store('octane')->decrement('my-number');
    return $number;
});

=====

Route::get('/save-many', function () {
    Cache::store('octane')->putMany([
        'my-number' => 42,
        'my-string' => 'Hello World!',
        'my-array' => ['Kiwi', 'Strawberry', 'Lemon'],
    ]);
    return "Items saved!";
});

Route::get('/get-many', function () {
    $array = Cache::store('octane')->many([
        'my-number',
        'my-string',
        'my-array',
    ]);
    return $array;
});

=====
```

```
Route::get('/get-one-from-many/{key?}', function ($key = "my-number") {
    return Cache::store('octane')->get($key);
});
```

=====

Swoole Table

If you want to share data across workers **in a more structured way than a cache**, you can use Swoole Table.

=====

If we want to use Swoole Table, we must define the table's structure. The structure is defined in the **config/octane.php** file in the tables section.

A table **named example with a maximum of 1,000 rows** (example:1000) with 2 fields. The first one is called **name and is a string** with a maximum of 500 characters, and the second field is called **votes, and the type is an integer** (int).

```
'tables' => [
    'example:1000' => [
        'name' => 'string:1000',
        'votes' => 'int',
    ],
],
```

=====

To create a table:

```
'tables' => [
    'example:1000' => [
        'name' => 'string:1000',
        'votes' => 'int',
    ],

    'my-table:100' => [
        'uuid' => 'string:36',
        'name' => 'string:1000',
        'age' => 'int',
        'value' => 'float',
    ],
],
```

=====

```

Route::get('/table-create', function () {
    // Getting the table instance
    $table = Octane::table('my-table');
    // looping 1..90 creating rows with fake() helper
    for ($i=1; $i <= 90; $i++) {
        $table->set($i,
            [
                'uuid' => fake()->uuid(),
                'name' => fake()->name(),
                'age' => fake()->numberBetween(18, 99),
                'value' => fake()->randomFloat(2, 0, 1000)
            ]
        );
    }
    return "Table created!";
});

```

```

Route::get('/table-get/', function () {
    $table = Octane::table('my-table');
    $row = $table->get(1);
    return $row;
});

```

=====

To get all the table values:

```

Route::get('/table-get-all', function () {
    $table = Octane::table('my-table');
    $rows = [];
    foreach ($table as $key => $value) {
        $rows[$key] = $table->get($key);
    }
    // adding as the first row the table rows count
    $rows[0] = count($table);
    return $rows;
});

```

=====

Metrics

Swoole offers a method to retrieve some metrics about the resource usage of the application server and the workers. This is useful if you want to track the usage of some aspect – for example, the time, the number of requests served, the active connections, the memory usage, the number of tasks, and so on. To retrieve metrics, first of all, you have to access the Swoole class instance. Thanks to the service container provided by Laravel, you can resolve and access objects from the container. The Swoole server object is stored in the container by Octane. Via **App::make**, you can access the Swoole server class instance.

```
=====
Route::get('/metrics', function () {
    $server = App::make(Swoole\Http\Server::class);
    return $server->stats();
});
=====
```

The Swoole server Stats object is returned as a response.

```
=====

To use the Octane routes:

Octane::route('GET', '/dashboard', function() {
    return new Response((new DashboardController)->index());
});

Octane::route('GET', '/dashboard-concurrent', function() {
    return new Response((new DashboardController)->indexConcurrent());
});
=====
```

To use the query scope:

```
class Event extends Model
{
    public function scopeOfType(mixed $query, string $type): mixed
    {
        return $query->where('type', $type)->where('description', 'LIKE',
'%something%')->orderBy('date')->limit(5);
    }
}

=====

public function indexConcurrent() {
    $time = hrtime(true);
    try {
        [$count, $info, $warning, $alert] = Octane::concurrently([
            fn () => Event::count(),
            fn () => Event::ofType('INFO')->get(),
            fn () => Event::ofType('WARNING')->get(),
            fn () => Event::ofType('ALERT')->get(),
        ]);

    } catch (TaskTimeoutException $e) {
        return response()->json([
            "msg" => $e->getMessage(),
        ]);
    }
    $totalTime = (hrtime(true) - $time) / 1_000_000;
    return response()->json([
        'count' => $count,
        'info' => $info,
        'warning' => $warning,
        'alert' => $alert,
        'totalTime' => $totalTime,
    ]);
}

=====
```

Route Caching: If you define custom routes via `Octane::route` or use dynamic routing, you should avoid using `php artisan route:cache`, as it will break these definitions.

=====

When performing HTTP calls concurrently, we should use `Route::get`:

```
Octane::route('GET', '/api/sentence', function () {
    sleep(1);
    return response()->json([
        'text' => fake()->sentence()
    ]);
});
Octane::route('GET', '/api/name', function () {
    sleep(1);
    return response()->json([
        'name' => fake()->name()
    ]);
});

Route::get('/httpcall/sequence', function () {
    $time = hrtime(true);
    $sentenceJson = Http::get('http://127.0.0.1:8000/api/sentence')->json() ??
[];
    $nameJson = Http::get('http://127.0.0.1:8000/api/name')->json() ?? [];
    $time = hrtime(true) - $time;

    return response()->json(
        array_merge(
            $sentenceJson,
            $nameJson,
            ['time_ms' => $time / 1_000_000]
        )
    );
});
Route::get('/httpcall/parallel', function () {
    $time = hrtime(true);
    [$sentenceJson, $nameJson] = Octane::concurrently([
        fn () => Http::get('http://127.0.0.1:8000/api/sentence')->json(),
        fn () => Http::get('http://127.0.0.1:8000/api/name')->json(),
    ])
    );
    $time = hrtime(true) - $time;

    return response()->json(
        array_merge(
            $sentenceJson,
            $nameJson,
            ['time_ms' => $time / 1_000_000]
        )
    );
});
```

```
});
```

```
=====
```